

Data Science @ scale 2/3

MLOPS – DELIVERY, ARCHITECTURES

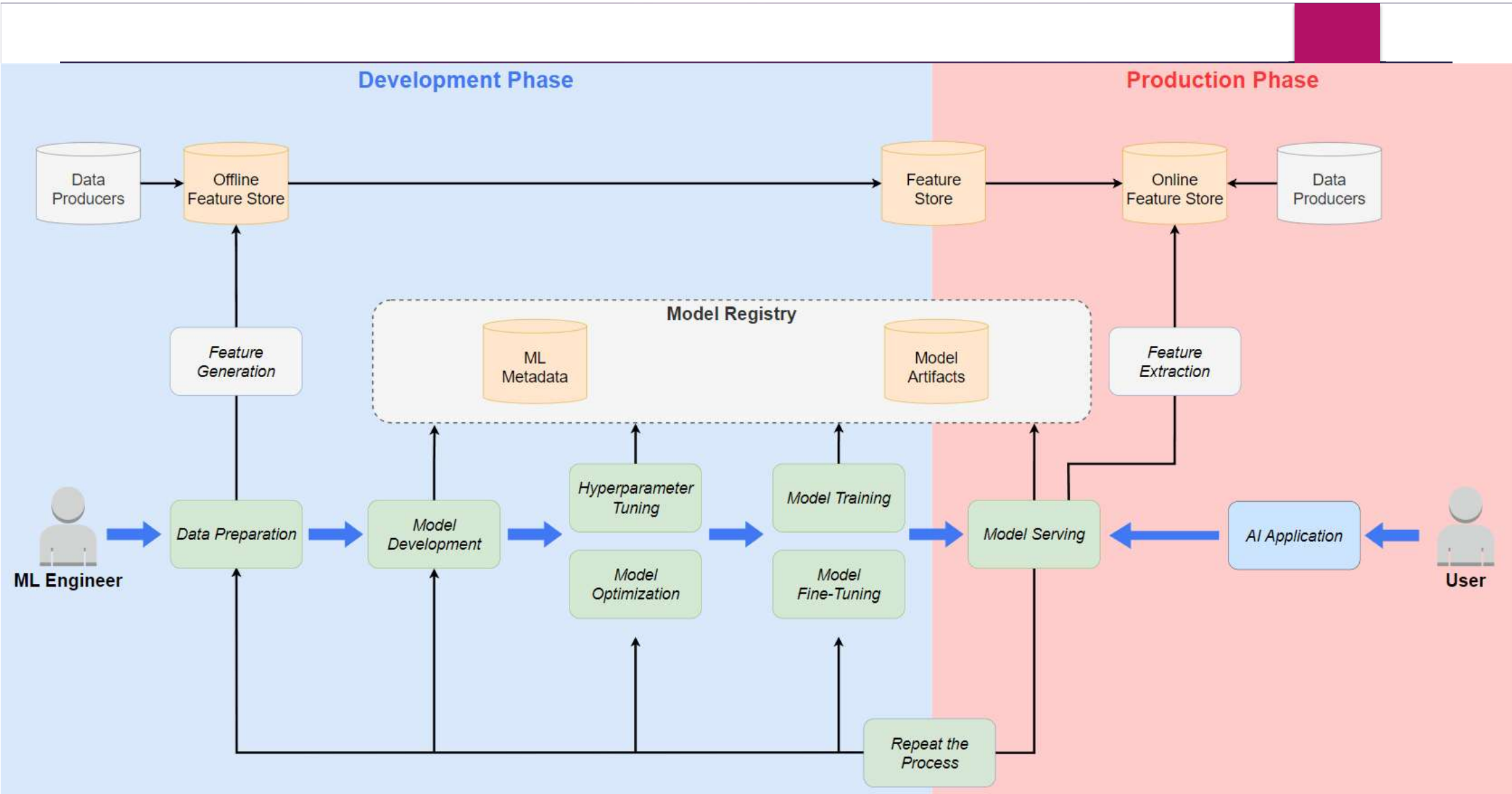
ALEXANDRE GARCIA – DAUPHINE EXECUTIVE MASTER

Motivation

- ▶ Part 1 was about **organization, roles, projects**: what is called **inner loop**
- ▶ Part 2 is about supporting inner loop for heavy tasks (training) and the **outer loop** of delivering safely and continuously to production
- ▶ **ML Ops operates Outer loop but also supports inner loop**

Summary

- ▶ xOps: DevOps, DataOps, MLOps, LLMOps
- ▶ GPUs, Optimizations
- ▶ Cloud
- ▶ Containers
- ▶ Containers Orchestration: Kubernetes
- ▶ Specific scenarios
- ▶ Vendors



xOps

- ▶ DevOps: quality of code (bugs, downtime)
- ▶ DataOps: quality of the data (freshness, quality, volume)
- ▶ MLOps: dynamic DevDataOps + adjust for very different workloads
 - ▶ train massively
 - ▶ serving: arbitration between response time and cost
 - ▶ Training/serving skew -> monitor performance
 - ▶ optimize resources and cost -> dynamic scaling, cloud
- ▶ AIOps: hybrid (dev, data, ml, external LLM apis)
- ▶ https://services.google.com/fh/files/misc/practitioners_guide_to_mlops_whitepaper.pdf

Model training

- ▶ Necessary for experiments and retraining when performance drops
- ▶ Workloads may require acceleration (GPU/TPU), huge storage (Hadoop) or memory for big data ingestion (Spark)
- ▶ Takes a long time: needs task orchestration, scheduling, fail policy
- ▶ Tools like Airflow, Prefect work with Python DAGs
- ▶ They were designed for data pipelines but can also schedule long running trainings
- ▶ Teaser: All of this is generally available on cloud solutions

Acceleration: GPUs...

- ▶ Nvidia: not only GPUs but drivers and libraries like Cuda, TensorRT (inference speed up), cuDNN(Neural Networks), NCCL (communication between GPUs), FLARE (distributed learning)
- ▶ Cuda is the low level library for executing code on GPUs: a function called kernel executed on GPU by M blocks $\times$ N threads = MN threads. Tricky to reach optimal throughput between CPUs and GPUs. Lot of tooling (Nvidia, Pytorchtraces)
- ▶ Numba is compatible with Cuda and an easier way to parallelize python code to machine code via LLVM
- ▶ AMD is compatible with CUDA
- ▶ ARM on devices, TPUs by Google => GPU/Pytorch vs TPU/Jax
- ▶ And we don't even consider inference...

Learning strategies

- ▶ Federated learning: different datasets learnt independently and aggregated -> NVidia FLARE, interesting for dataset sensitivity
- ▶ Curriculum Learning: from easy to complex (order training data) a bit like boosting
- ▶ Q-Learning: for reinforcement, model-free, off-policy, State-Action matrix
- ▶ <https://www.comet.com/site/blog/pretraining/> Continuous pre-training, interleaved

CICD: way to outer loop

- ▶ Continuous Integration, Continuous Deployment
- ▶ Toggle feature for smooth delivery
- ▶ Criteria on accurate testing and code quality
- ▶ Everything should be git-centric (GitOps)
- ▶ Automatic version number from code (Semantic versions, conventional commits)
- ▶ Different environments with promotion: dev/uat/next/prod
- ▶ Model registries should also span pre and post release
- ▶ <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>

Cloud Providers

- ▶ Clouds: Cloud Services Providers (CSPs), hybrid (RedHat OpenShift), OSS (OpenStack)
- ▶ IaaS, PaaS, SaaS
- ▶ Directories, identity federation
- ▶ Standard solutions available on the shelf and managed
- ▶ Availability zones
- ▶ Cost, alerting
- ▶ LLM integration

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking

 You manage

 Service provider manages

Cloud Setup

- ▶ **Hub and Spoke**
- ▶ Hub is controlled by central cloud team in the company: you cannot use everything
- ▶ Spokes are created with specific rules for each entity
- ▶ Private links to enable specific accesses between spokes
- ▶ **Subscription** is the billing information and the root for creating resources
- ▶ Cost can be controlled with budgets and automatic alerts
- ▶ You can have more than 1 cloud solution: internal, external...

Cloud Storage

- ▶ AWS S3
- ▶ More generally cloud storages are apis
- ▶ Api means storage is extensible
- ▶ Generally blob storage but you can mount file storage(file shares), table storage
- ▶ On Azure there is a specialized blob storage for Datalake with Hadoop HDFS compatibility (Spark is compute on Hadoop storage)

Cloud Compute

- ▶ Batch
- ▶ Lambda (serverless)
- ▶ Kubernetes clusters
- ▶ Node pools (size of node)
- ▶ You can set basic requests and limits for the Kubernetes workloads
- ▶ Long tasks may require asynchronism and middleware
- ▶ Queues, Celery

Cloud AI

- ▶ Major Cloud Providers have deals with model providers: Azure (OpenAI), AWS (Anthropic)
- ▶ Google (GCP) have their own HPC solution with TPUs: optimized for NN training and low electric consumption
- ▶ Setup token limits, bandwidth
- ▶ Vector DBs

Typical components

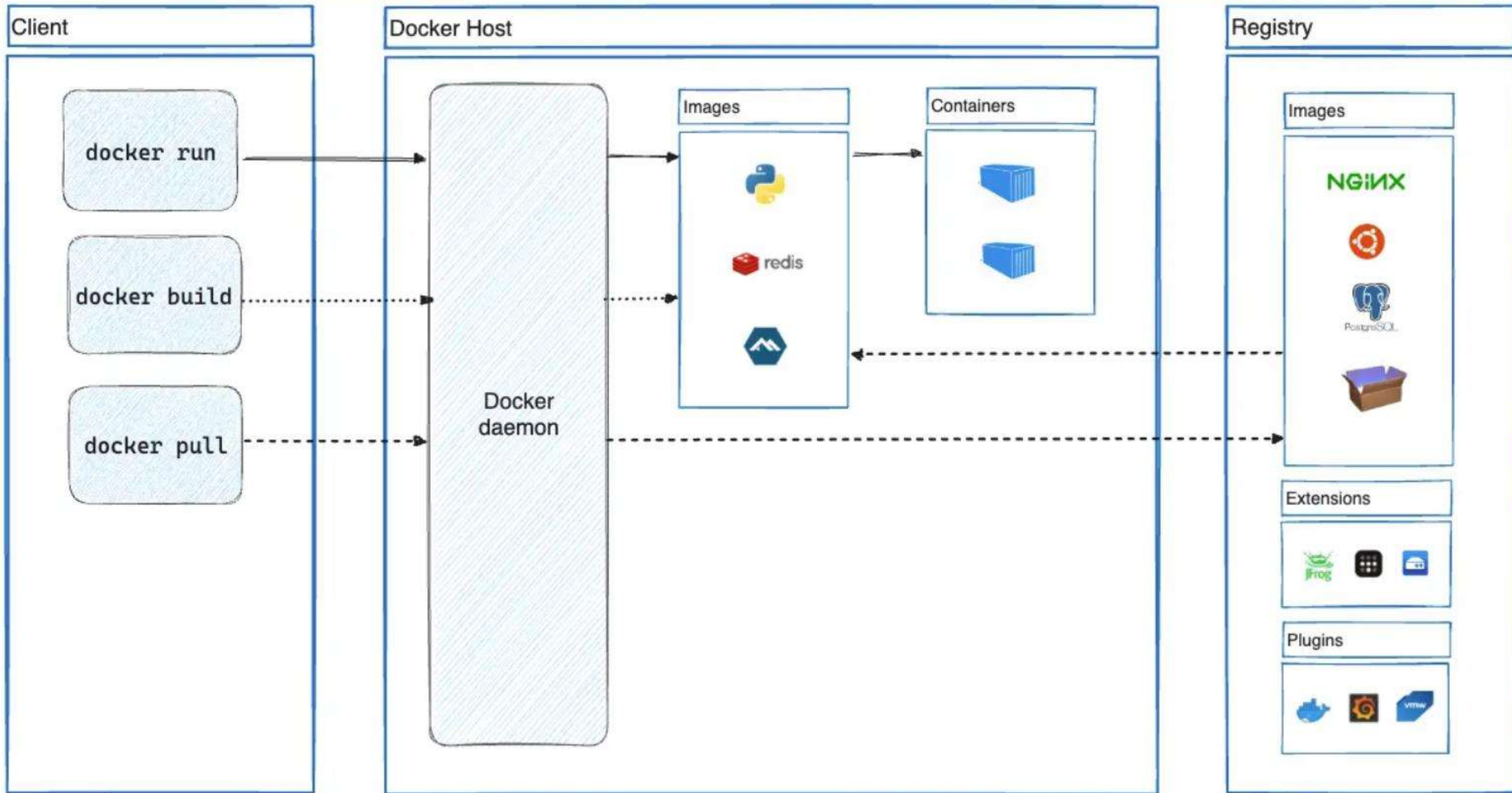
- ▶ Feature store: depends on cost of crafting features and latency requirements, offline/online
- ▶ Spark engines
- ▶ Model registry
- ▶ Checkpointers for model weights
- ▶ Checkpointers for agent states
- ▶ Vector DB
- ▶ AI observability (traces, correlations through all system)

OS

- ▶ **Linux** is the standard: Resources, Cost, Lesser editor lock
- ▶ Many Linux distributions:
 - ▶ Fedora/RedHat/CentOS -> RedHat main commercial distribution
 - ▶ Debian/Ubuntu -> Ubuntu is the choice on Azure
 - ▶ Alpine, ... -> very light OS for « small » processes
- ▶ Beside LTS versions, OS must be constantly maintained: vulnerability checks (Trivy) before deployment
- ▶ For development: MacOS, WSL2 (Linux by Microsoft on Windows), Remote Linux from vscode/pycharm
- ▶ Advice: learn some basic Linux command line

Containers

- ▶ Docker is the pioneer/leader but OCI (Open Container Initiative) standard
- ▶ Images, Registries, Engines
- ▶ A container is a lightweight program created by an engine that pulls program code from an image stored in a remote registry.
- ▶ Key points:
 - ▶ containers are isolated
 - ▶ Files for execution are distributed in the images: no prerequisites
- ▶ Registry: dockerhub, harbor
- ▶ Engines: docker, podman, kaniko



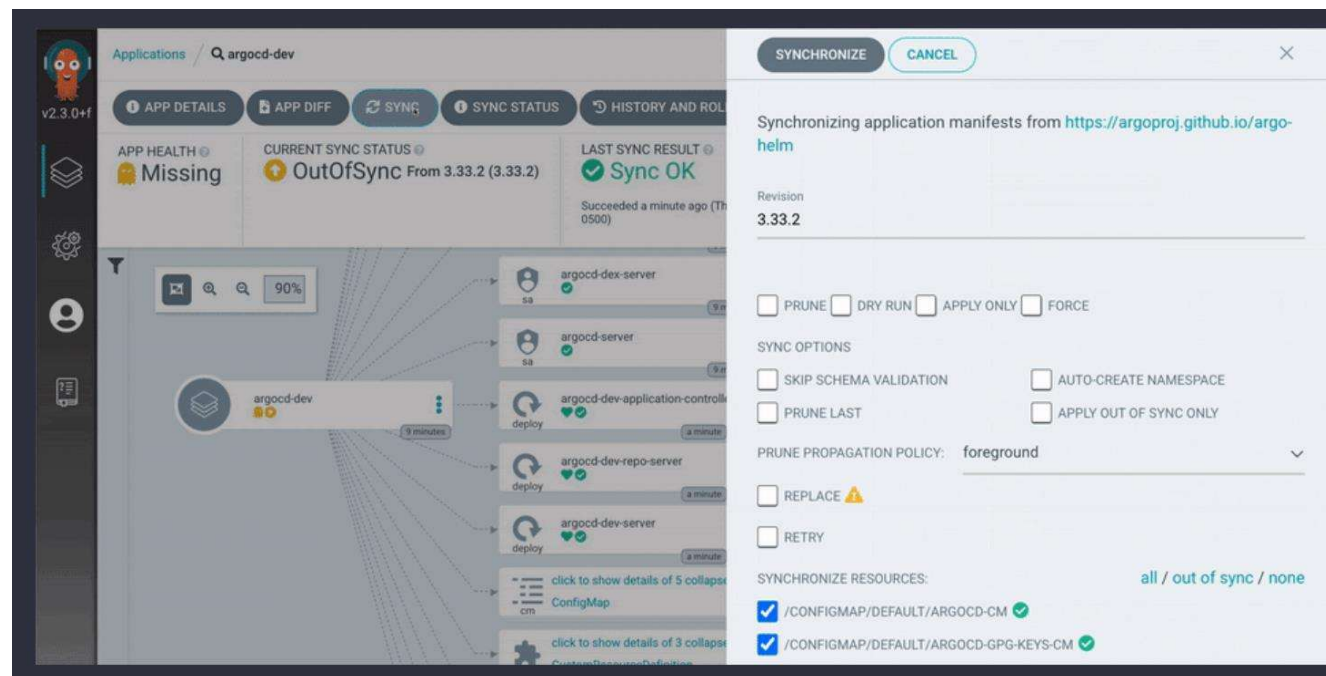
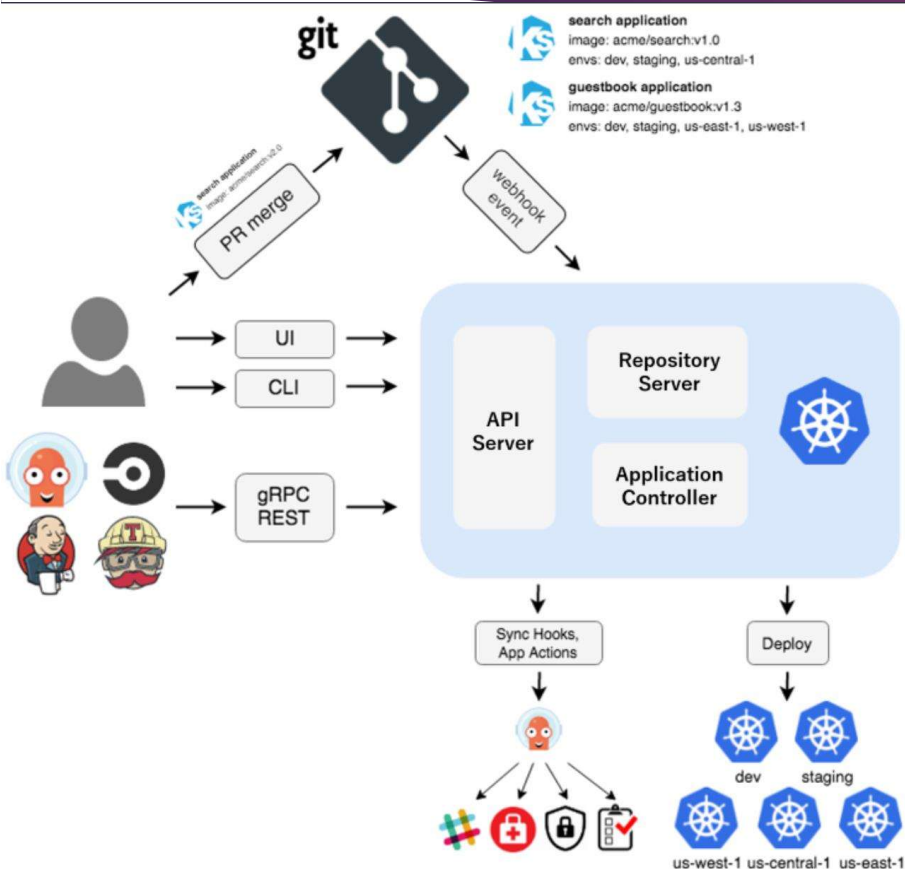
Application deployment

- ▶ Deployments are actually more complex than a simple program.
 - ▶ More than 1 program
 - ▶ Databases, middleware, api
 - ▶ Replica for redundancy
 - ▶ Network description
 - ▶ Which resource requirements for the machines (CPU, Memory, Geography) ?
- ▶ So there is a need for:
 - ▶ specific languages (Helm, Terraform, OpenTofu): very configuration oriented
 - ▶ a whole infrastructure manager

Containers Orchestration

- ▶ Kubernetes(k8s) is the standard specification to operate deployments
 - ▶ implemented by all cloud providers (eg Microsoft: Azure Kubernetes Services)
 - ▶ Major feature is auto scaling (basic or on metrics: KEDA), monitoring, apis
 - ▶ Dashboards through CSP portals or OpenSource (Lens)
- ▶ Very important: Kubernetes is extensible (accepts custom resources from external projects)
- ▶ Kubernetes operators

ArgoCD: git centric K8S



Kubeflow

- ▶ A Kubernetes toolkit for AI
 - ▶ Pipelines, Spark operator
 - ▶ Notebooks, Code-server
 - ▶ Dashboard
 - ▶ Model training (Kubernetes operator)
 - ▶ AutoML
 - ▶ Model serving

Maintenance requirements

- ▶ Have automated user feedback, usage monitoring (Elastic Search, Kibana, Graphana, enterprise frontend)
 - ▶ Technical metrics for the platform
 - ▶ Traces, correlation ids Observability and evaluation:
<https://www.comet.com/site/blog/llm-observability-tools/>
 - ▶ Monitor data quality problems and drift
- ▶ Online experimentation: A/B testing, multi-armed bandits
 - ▶ typical in marketing: Digital Experience Platforms (DXP) like Optimizely, VWO

Quantization, distillation, fine tuning

- ▶ To take less space, there are techniques to reduce model size: quantization for size of parameters
- ▶ Knowledge Distillation is another process: have a large model and weak learners that learn on same data plus large model output probas (class/token)
- ▶ LORA, PEFT
- ▶ Foundation Models: Time Series (TimesFM), Transaction (NVidia)
- ▶ Main tasks: Pre Training, Post Training (Fine Tuning, RLHF)

RAG

- ▶ Different kinds: Hybrid...
- ▶ Vectorial DBs, flat indexes, reduce dimensions or search scope
- ▶ <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>
- ▶ Specific metrics: Context Recall, RAGAS
- ▶ ANN: HSNW <https://www.pinecone.io/learn/series/faiss/vector-indexes/>

Graphs

- ▶ Neo4j
- ▶ Pytorch Geometric: GNN, GAT
- ▶ Specific features: fraud detection, ontologies, GraphRAG

Specific deployment use cases

- ▶ Inference requires generally an API to wrap models and fulfill company rules (identity) but you can have more integration (Azure ML) to benefit from frontends, ops
- ▶ For specific business requirements, we can deploy an API and even a front end for user acceptance
 - ▶ FastAPI is the reference for Python APIs
 - ▶ Streamlit is widely used for prototyping without HTML or Javascript skills (competitors: Plotly Dash, Bokeh, Nicegui). Voila can webify a Jupyter notebook.
 - ▶ Agents: Chainlit, Gradio
- ▶ Additional components: OCR, external APIs ...

ML vendors solutions

- ▶ Those solutions offer various deployment plans (cloud, on-prem)
- ▶ Dataiku: all purpose data roles, can generate api and streamlit frontends, data pipelines
- ▶ Weights and Bias: <https://wandb.ai/site/for-enterprise/deployment-options/> strong on experiments and deep learning
- ▶ Databricks and Azure leverage on MLFlow
- ▶ Under the hood they use about the same technologies we'll describe next, so interesting to delve into it even in a vendor setup

Conclusion for ML Ops

- ▶ DataScientist and MLOps are different roles but with a clear mutual interest to cooperate
- ▶ Many solutions for data/dev/ops, many skills needed -> MLOps maturity can vary wildly
- ▶ Profiles with good fluency ML+MLops are bankable on the market
- ▶ Cloud is not 100% mandatory, Git/Containers/Kubernetes are
- ▶ GPU comes with overhead (code, infra, cost)
- ▶ AI/LLMOps expected to be a mix with evaluation methods yet unclear